UTM

UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER II, SESSION 2025/2026

SECJ3483 01

WEB TECHNOLOGY

TITLE :

SECURE PERSON BMI FULL-STACK WEB APPLICATION

PREPARED BY:

NAME	MATRIC NUMBER
NURIN IZZATI BINTI MOHD RASHIDIN	A23CS0161
SHAJANNATUL IMAN BINTI ABDUL MAJID	A23CS0267
INTAN SERINA BINTI ANUAR MUS	A23CS0085
ASILAH BINTI MOHD RAZAK	A23CS0051

Table Of Contents

Table Of Contents **2**
1.0 Security Investigation Summary **3**
1.1 Investigation Table 3
2.0 Weaknesses Found **5**
2.1 Weakness Classification Table 5
3.0 Fixes Implemented **6**
3.1 Problem-Solution Mapping Table 6
4.0 Testing Evidence **7**
4.1 Before and After Testing Evidence 7
5.0 API Endpoints Tested **10**
5.1 Public routes 10
5.2 Protected user routes 10
5.3 Staff routes 10
5.4 Admin routes 11
6.0 GitHub Commit Summary **12**
6.1 Version Control Log 12
7.0 Reflection Answers **13**

1.0 Security Investigation Summary

1.1 Investigation Table

Test No	What I Tested	Expected Secure Behavior	Actual Result	Is It a Weakness ?	Why?
1	Add BMI with invalid data	Backend should reject invalid data	Backend accepts the negative numbers and empty names which corrupt the database.	Yes	The backend trusts the frontend blindly as the users could bypass the Thunder Client.
2	Access another user's BMI record	Backend should deny access	Backend returns the another user's data.	Yes	The route checks only whether a token exists, not whether the logged in user owns the BMI record
3	Access staff route as normal user	Backend should deny access	Normal user can access the staff route	Yes	The staff route only checks for a valid token, not the user role.

4	Access staff route as normal user	Backend should deny access	Normal user can access the staff route	Yes	The admin route only checks for a valid token, not the user role.
5	Submit XSS payload in notes	Payload should display as text	Payload is stored and may be rendered by the frontend	Yes	The backend does not sanitize notes before saving
6	Check API response	Password/hash should not be visible	Login returns the full user record, including password and password_hash	Yes	Sensitive fields are exposed in the API response.
7	Try SQL Injection input	Input should be treated as data	The attack input bypassed the password check and the attacker can logged in successfully.	Yes	The code combines user input into the database command which allows the attacker to change how the query

					works.
8	Try to update protected fields	Backend should reject or ignore protected fields	User can update protected fields such as user_id, bmi, and category	Yes	Mass assignment is allowed; the backend trusts client-supplied fields too much.

2.0 Weaknesses Found

2.1 Weakness Classification Table

Weakness Found	Frontend / Backend / API / Database	Security Category	Possible Risk
Negative weight accepted	Backend	Missing backend validation	Invalid data stored
User accesses another BMI record	API / Backend	Broken Object Level Authorization	Privacy violation
Normal user accesses admin route	API / Backend	Broken Function Level Authorization	Unauthorized admin action
API returns password_hash	API / Backend	Sensitive data exposure	Credential risk
XSS payload executes	Frontend	XSS	Browser script execution
SQL Injection input affects query	Backend / Database	SQL Injection	Unauthorized data access
User changes user_id or role	API / Backend	Mass assignment	Privilege or ownership manipulation

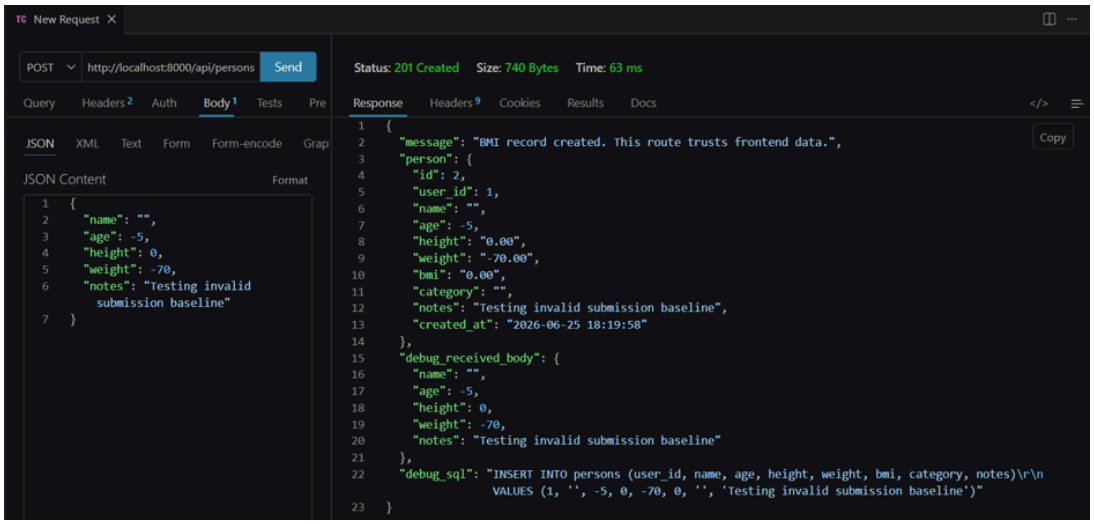
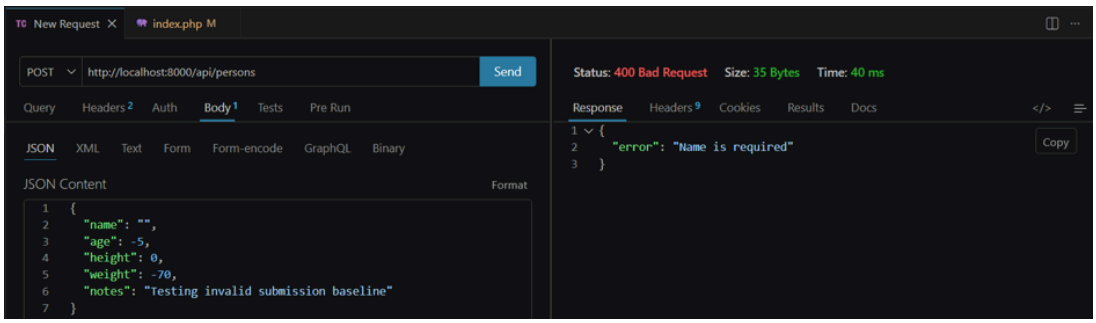
3.0 Fixes Implemented

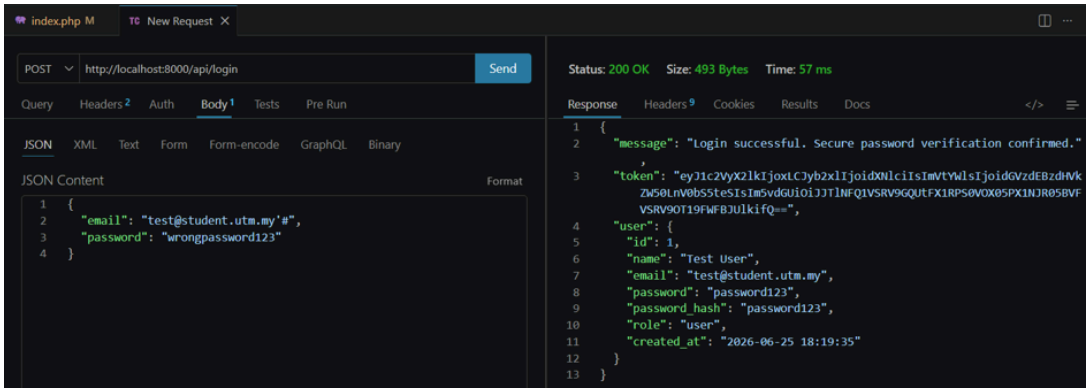
3.1 Problem-Solution Mapping Table

Problem	Root Cause	Where to Fix?	Proposed Solution	How to Retest?
Invalid BMI accepted	Backend does not validate	Backend	Validate name, age, height, weight	Submit invalid data again
User accesses other record	No ownership check	Backend API	Compare JWT user_id with record user_id	Access other user's record
User accesses admin route	No role check	Backend API	Check role from verified JWT	Use user token on admin route
password_hash returned	API returns all fields	Backend API	Select only safe fields	Check JSON response
XSS executes	Unsafe rendering	Frontend	Use {{ }} or textContent	Submit XSS payload again
SQL Injection possible	String SQL query	Backend	Use prepared statement	Try injection input again
user_id can be changed	Blind update	Backend API	Allow only safe fields	Send user_id in PUT request

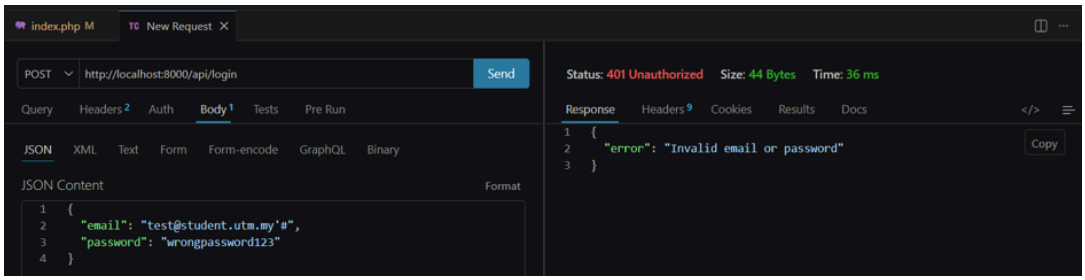
4.0 Testing Evidence

4.1 Before and After Testing Evidence

No.	Test Case
1	**Add BMI with negative weight and submit empty name** Before Fix: Accepted  After Fix: 400 Bad Request 
2	**SQL Injection login input** Before Fix: May affect query

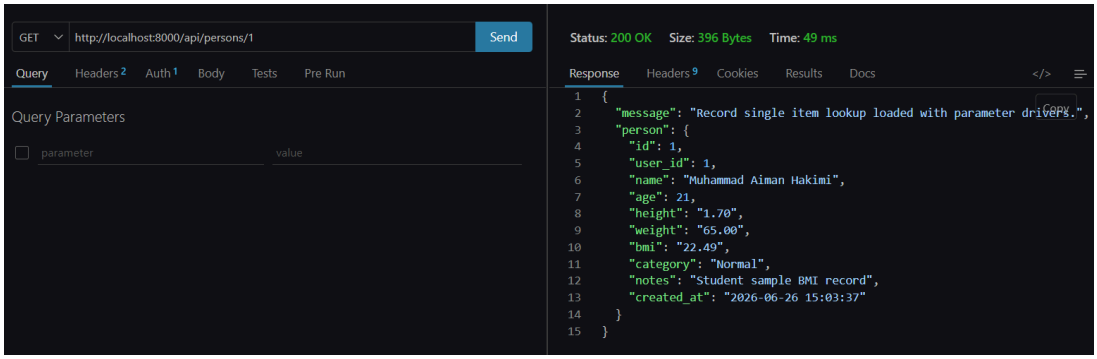


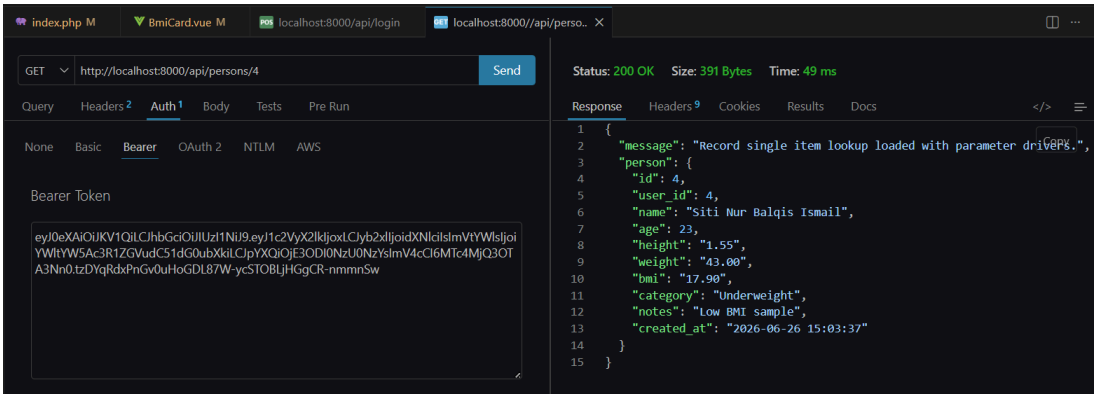
After Fix: Treated as data



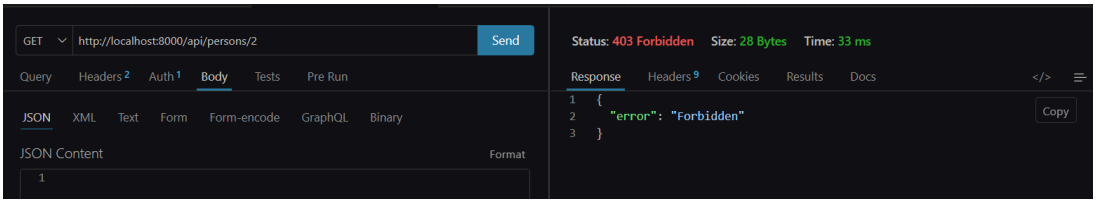
3 **Access another user's BMI record**

Before Fix: Data returned



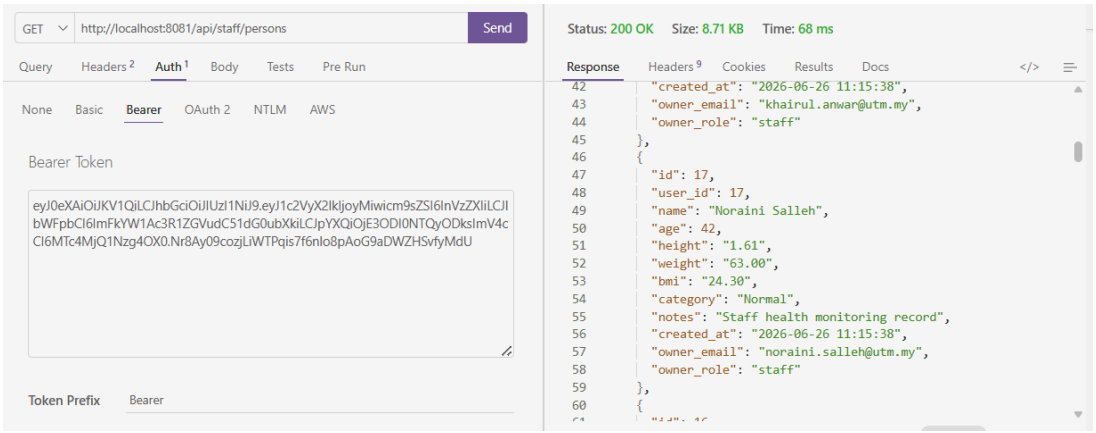


After Fix: 403 Access denied

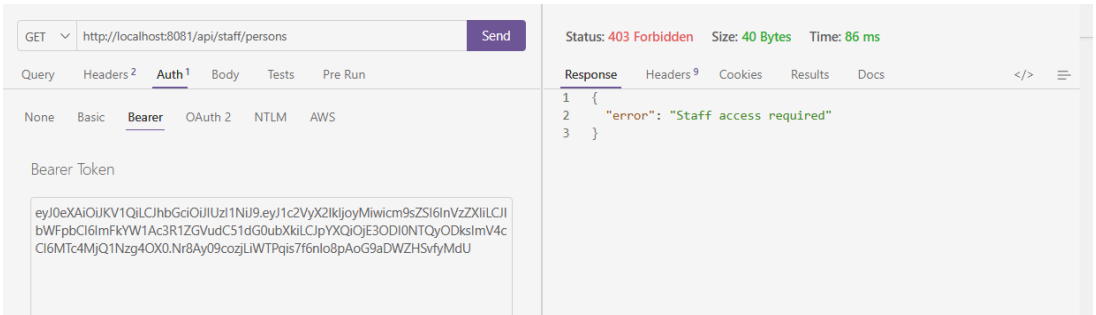


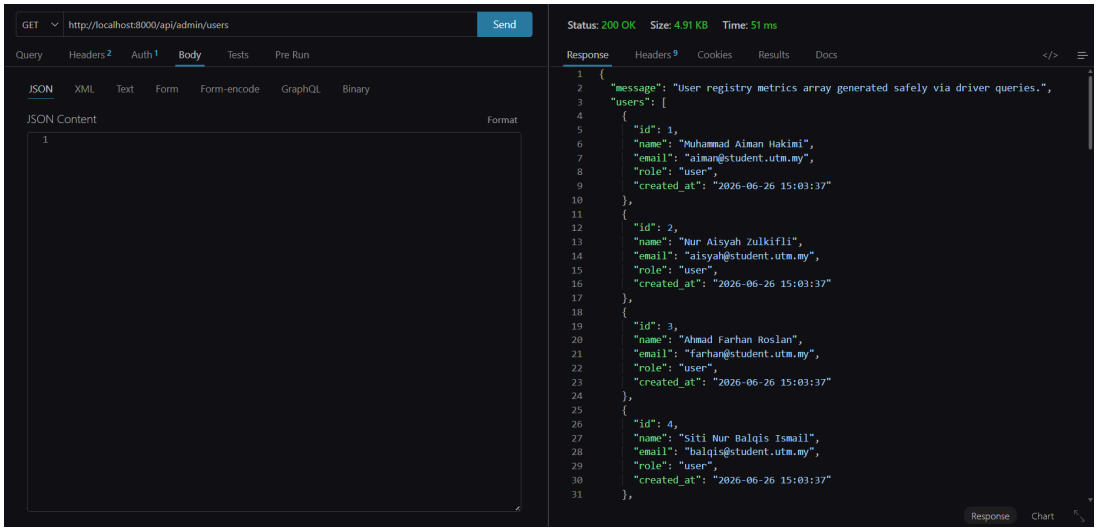
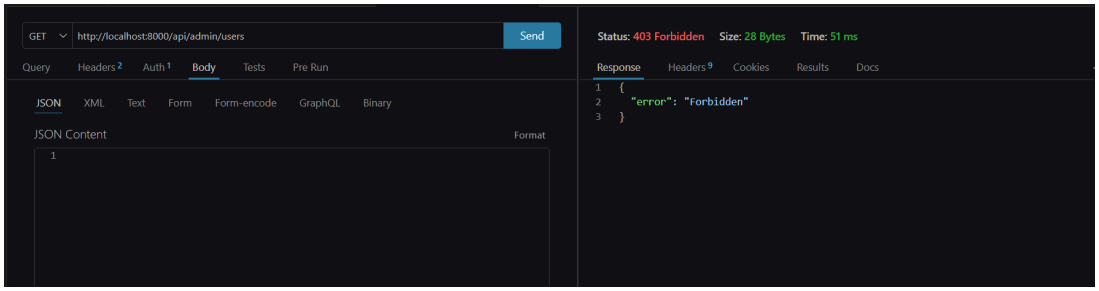
4 **Access staff route as user**

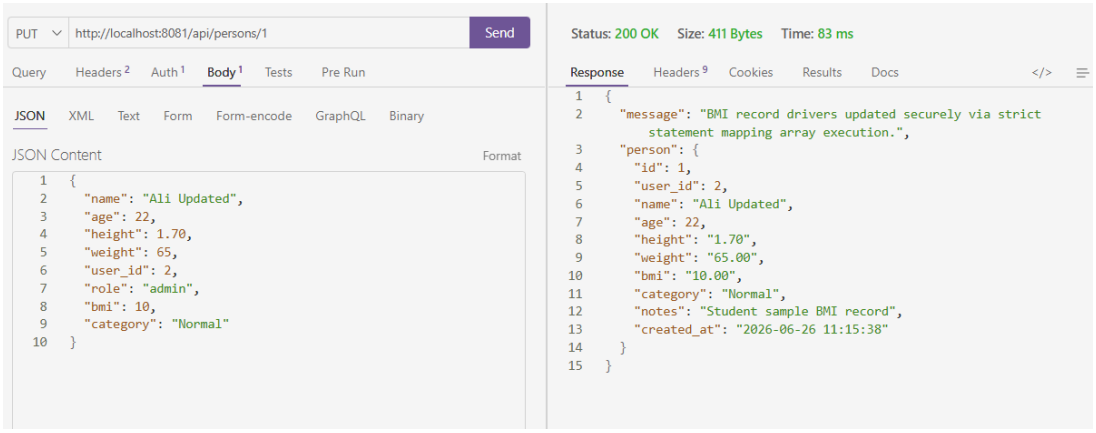
Before Fix: Allowed



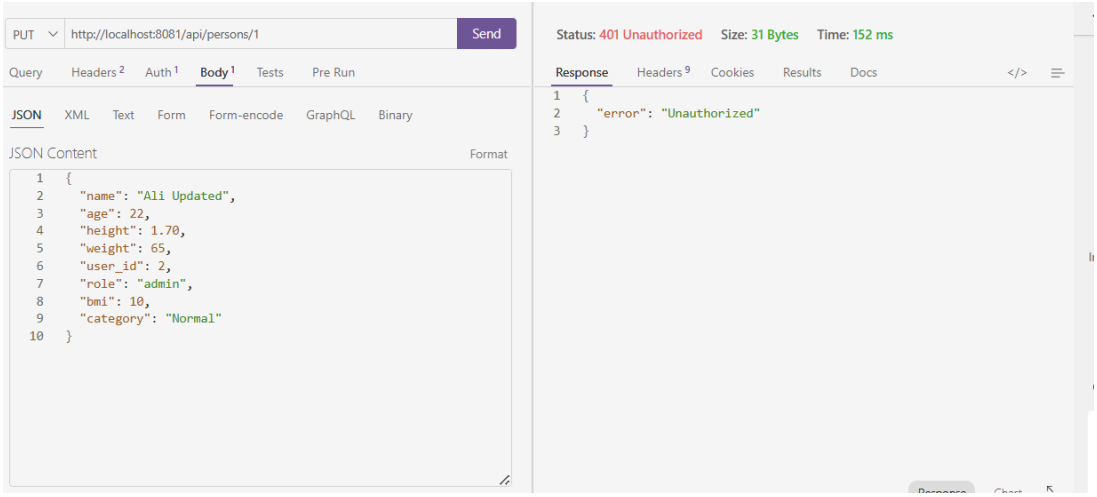
After Fix: 403 Staff access required



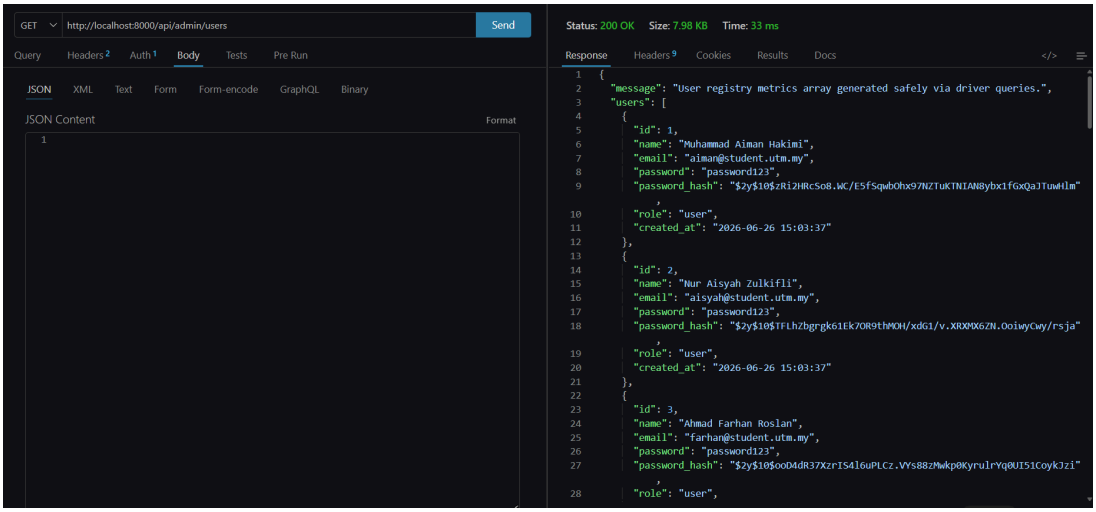
5	**Access admin route as user** Before Fix: Allowed  After Fix: 403 Admin access required 
6	**Update user_id or role** Before Fix: Accepted



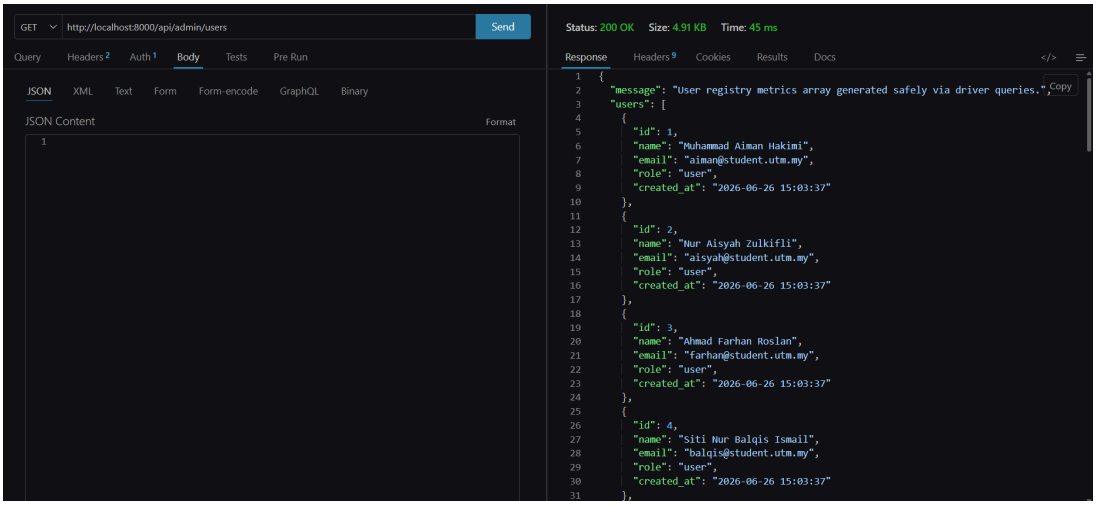
After Fix: Rejected or ignored



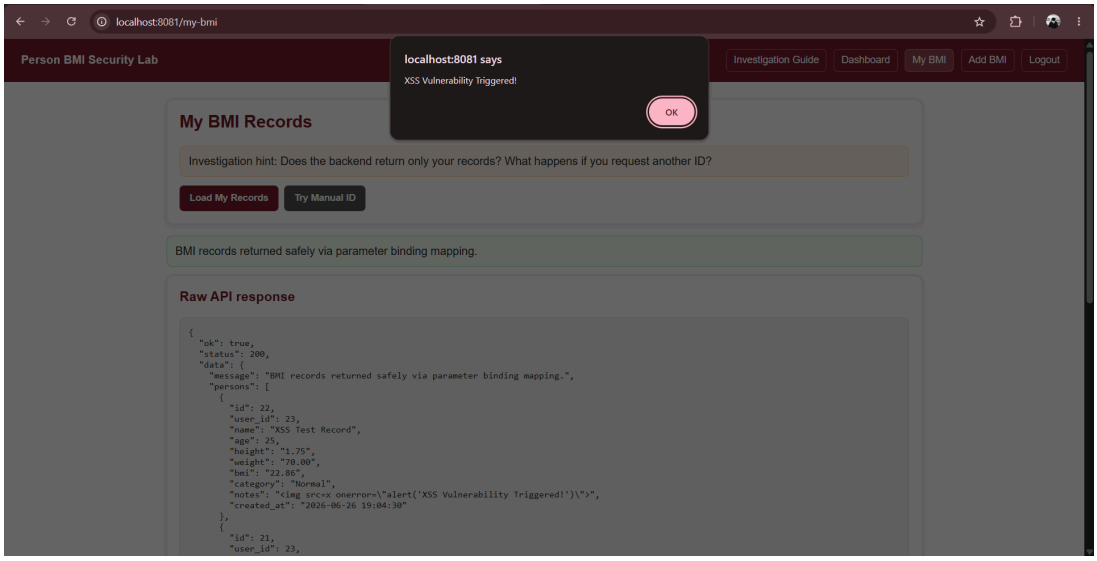
7	**API returns password_hash** Before Fix: Visible



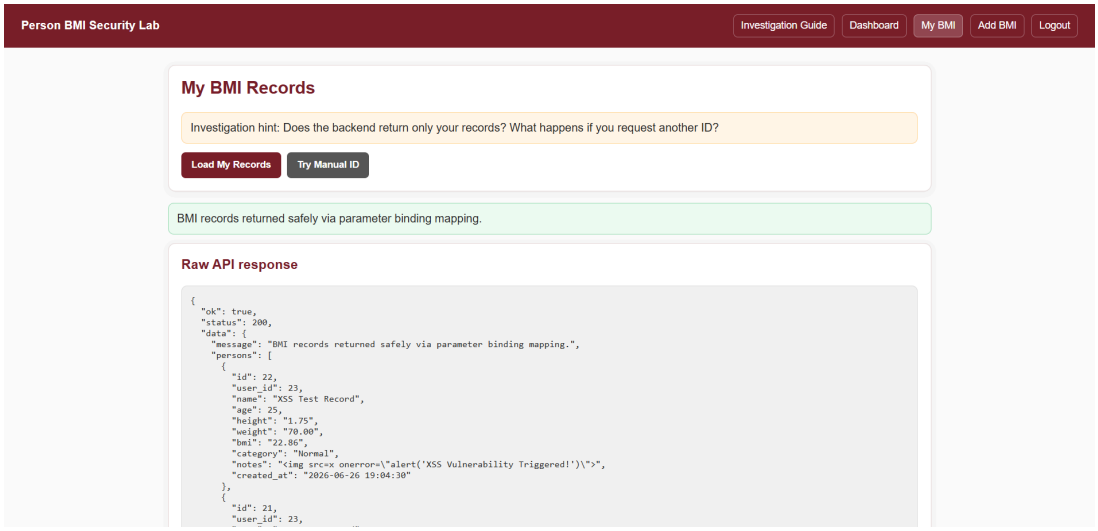
After Fix: Removed



8	**XSS payload in notes** Before Fix: Alert executes



After Fix: Displayed as text



5.0 API Endpoints Tested

5.1 Public routes

Method	Route	Purpose
POST	/api/register	Register new user
POST	/api/login	Login and receive JWT

5.2 Protected user routes

Method	Route	Purpose	Access
GET	/api/profile	View own profile	Any logged-in user
GET	/api/persons	View own BMI records	Owner only
POST	/api/persons	Add BMI record	Logged-in user
GET	/api/persons/{id}	View one BMI record	Owner, staff, admin
PUT	/api/persons/{id}	Update one BMI record	Owner or admin
DELETE	/api/persons/{id}	Delete one BMI record	Owner or admin

5.3 Staff routes

Method	Route	Purpose	Access
GET	/api/staff/persons	View all BMI records	Staff or admin

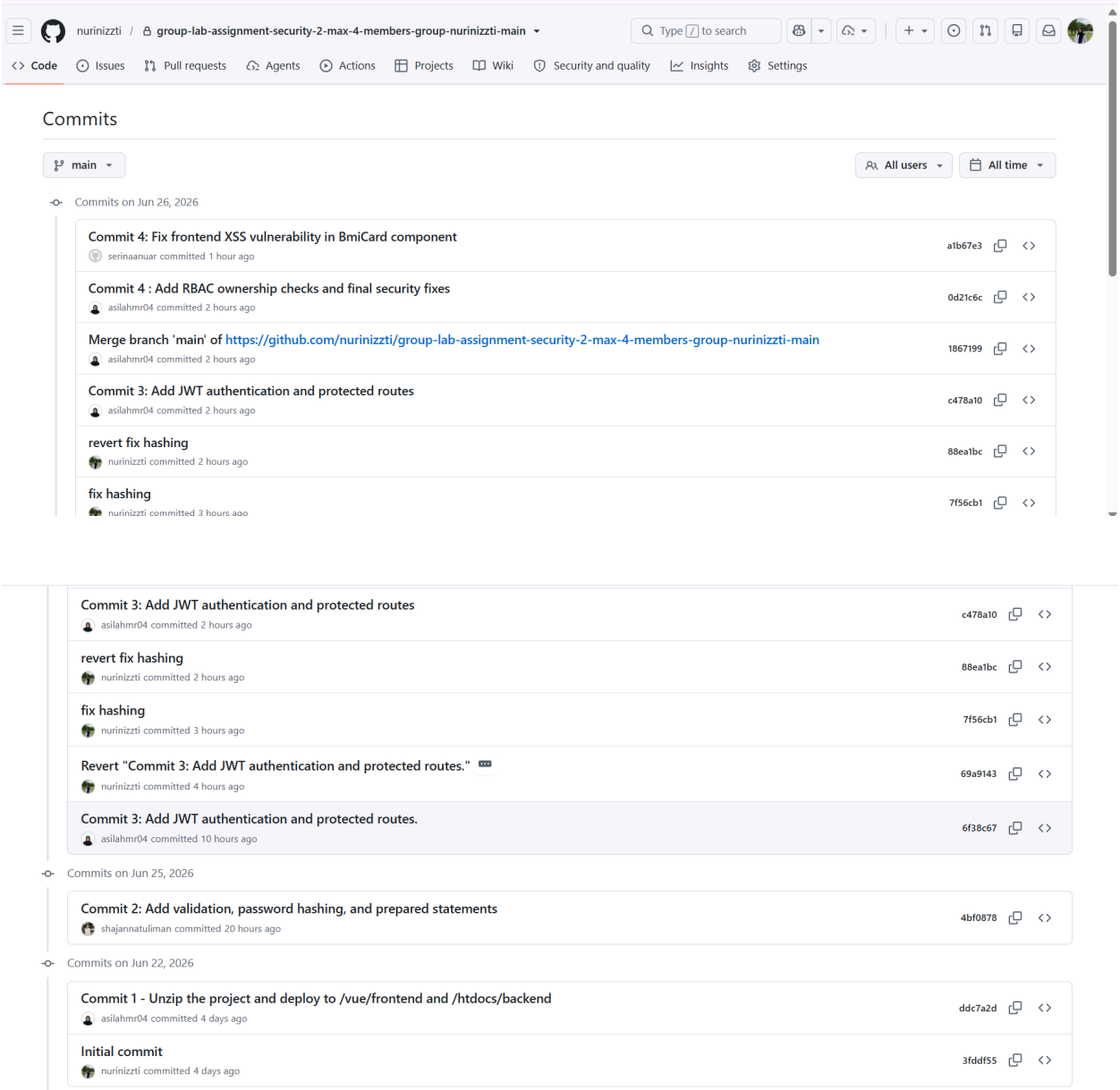
GET	/api/staff/persons/{id}	View any BMI record	Staff or admin

5.4 Admin routes

Method	Route	Purpose	Access
GET	/api/admin/users	View all users	Admin only
PUT	/api/admin/users/{id}/role	Change user role	Admin only
DELETE	/api/admin/persons/{id}	Delete any BMI record	Admin only

6.0 GitHub Commit Summary

6.1 Version Control Log



7.0 Reflection Answers

1. Which weakness was easiest to find? Why?

Broken Object Level Authorization (BOLA) or Broken Function Level Authorization (BFLA). Because they require no advanced hacking tools. Anyone can find them simply by changing an ID parameter integer in the URL such as from /api/bmi/5 to /api/bmi/6 or by calling a privileged endpoint directly using an unauthorized account token.

2. Which weakness was most dangerous? Why?

SQL Injection or Secret/Sensitive Data Exposure. Because SQL Injection allows a malicious actor to fundamentally change the structural meaning of a query to bypass login authentication or manipulate backend tables. Similarly, exposing backend signing secrets or database passwords compromises the entire application infrastructure globally.

3. Why is frontend validation not enough?

Frontend execution runs entirely on the client side and is fully transparent and visible to users. Attackers can easily deactivate client-side JavaScript restraints, rewrite HTML using browser DevTools, or maybe submit malicious payloads bypassing the frontend completely using API tools like Postman or Thunder Client. Frontend validation improves user experience, but only backend validation protects the system.

4. Why should BMI calculation be performed at the backend?

To enforce data integrity and prevent business logic manipulation.

5. What is the difference between authentication and authorization?

They address entirely different access verification stages. Authentication verifies a user's absolute identity like questioning "Who are you?". For example, validating an email and password hash combo. Authorization checks specific resource permissions

which ask "What are you allowed to do?". This is to ensure a logged-in user cannot execute administrative or cross-user actions.

6. What is the difference between RBAC and owner-based access control?

They partition authorization access control differently. Role-Based Access Control (RBAC) restricts access based on predefined privilege classes like separating permissions strictly between a user, staf, or admin. Owner-Based Access Control checks specific data relationships, verifying if the unique ID of the currently authenticated token match-pairs with the owner ID of the record being requested.

7. Why is JWT alone not enough to protect all data?

A JWT is explicitly designed to handle identity proofing, not situational object control.

8. Why should API responses exclude password_hash?

To prevent sensitive data exposure and system-wide credential exploitation.

9. Why is Vue route guard not enough to protect backend routes?

Vue route guards operate purely within the client-side user interface.

10. How did you prove that your security fix works?

By capturing comparative request evidence before and after implementing secure coding practices.